

Capstone Project Diary: VoxGest

Comprehensive Progress Report

Reporting Period: March 30, 2026 – April 24, 2026

1. EXECUTIVE SUMMARY

This document outlines the engineering progress, architectural decisions, and technical milestones for the VoxGest application from project inception (March 30) to the current application-ready state (April 24). Over this period, the machine learning pipeline evolved from a static localized hand-tracker to a highly robust, 162-feature Holistic Spatial-Temporal network. The project has successfully stabilized live webcam inference, completed real-time demonstrations, and executed a formal handoff to facilitate parallel mobile development.

2. MARCH 30 – EARLY APRIL: FOUNDATION & CORE RATIONALE

2.1. The One-Handed Accessibility Mandate (March 30)

Approach: VoxGest was conceptualized as an offline, real-time communication tool requiring only one hand for sign language input.

Reasoning: Research into the Deaf community revealed that one-handed ASL is widely practiced by necessity. Individuals who are Deaf and also possess a physical disability (e.g., stroke paralysis, amputation, or limb difference) rely exclusively on one-handed signing. By constraining our model to single-hand recognition, we intentionally adopted a stronger accessibility position than standard two-handed systems, ensuring we do not exclude this significant subset of the community.

2.2. Establishing the Evaluation Framework (Early April)

Approach: We finalized our evaluation parameters to align strictly with ISO/IEC 25010 standards and prepared a 5-point Likert scale questionnaire.

Reasoning: To ensure the system is objectively validated by both IT professionals and the Deaf/Mute community, we needed standardized metrics. Focusing on *Functional Suitability*, *Performance Efficiency*, *Reliability*, and *Usability* guarantees that our final capstone evaluation measures real-world communication viability, not just raw machine learning accuracy.

3. MID-APRIL: ARCHITECTURAL UPGRADES (APRIL 15 – APRIL 23)

3.1. Addressing the "Floating Hand" Blindspot

Approach: Transitioned the dynamic word model from *MediaPipe Hands* (63 features, wrist-normalized) to *MediaPipe Holistic* (162 features, nose-normalized).

Reasoning: Live testing revealed the model confused body-referential signs (e.g., WATER at the chin vs. PAIN at the chest). Wrist-normalization stripped away spatial context. By making the user's nose the geometric origin '(0,0,0)', the AI gained the mathematical ability to differentiate where the hand was located relative to the signer's body, drastically reducing class confusion.

3.2. Solving Live-Feed Inference Constraints

Approach: Implemented zero-padding for missing hand frames and integrated a standard-deviation "Motion Gate" into the live webcam script.

Reasoning: Continuous live video feeds suffer from "Temporal Segmentation" (the AI hallucinating words during idle moments or dropping sequences if the camera flickers). Zero-padding maintains sequence integrity during brief occlusions, while the Motion Gate mathematically blocks the AI from making predictions if the physical velocity of the hand is below '0.005', silencing false positives when the user is sitting still.

4. APRIL 24: LIVE STABILIZATION & MOBILE HANDOFF

4.1. Live Pipeline Demo & Sanity Checks

Approach: We executed safe checks on the codebase and performed manual webcam captures for a refined 5-word core target (YES, NO, PLEASE, WATER, and HELLO) before retraining the LSTM.

Reasoning: Before initiating mobile app integration, it was crucial to verify if the 5-word setup was stable enough to serve as a foundation. Recording real manual samples for these specific words proved that live dynamic word recognition was fully functional in a real-world webcam environment, moving the project from "trained scripts" to a verified pipeline.

4.2. The "App-Ready" Developer Handoff

Approach: We formally shifted focus from "model-only" development to an "app-ready hand-off." This involved creating `ANDROID_DEVELOPER_HANDOFF.md` and `runtime_manifest_v1.json`.

Reasoning: The Android developer required a clear, exact specification to integrate the '.tflite' models. The manifest explicitly details model inputs, outputs, labels, thresholds, and expected runtime behavior. This decouples the ML pipeline from the mobile UI, allowing the Android engineer to build the application independently and accurately.

4.3. Vocabulary Expansion Plan (Demo 10 Profile)

Approach: We added a new `demo10` profile in `word_config.py`, establishing the next 5 target words: HELP, STOP, DOCTOR, NAME, and THANKYOU.

Reasoning: This decision enables parallel development. The Android developer can immediately begin building the mobile application using the verified 5-word model, while the ML team concurrently gathers data and trains the 10-word profile. Because the runtime behavior is locked via the manifest, future vocabulary expansions will not block or break the mobile development process.